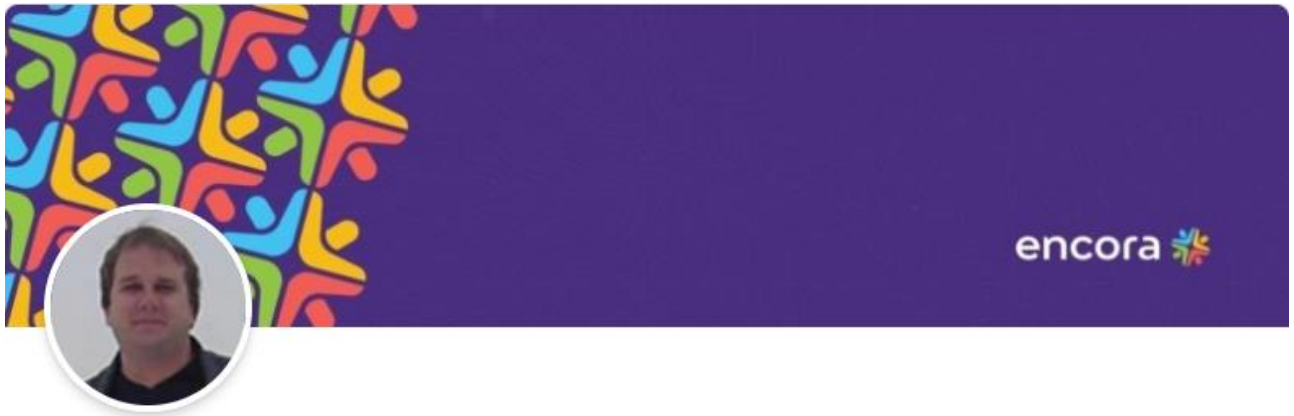
DAITAN IS NOW
encora

Puppet Debugger

Campinas, 06Mai22

Puppet Debugger

About myself



Roberto Nogueira

Software/Telecommunications Engineer

São José dos Campos, São Paulo, Brazil · 500+ connections

Join to follow

InMov

Universidade de São Paulo

Blog

Systems Specialist at Daitan (now Encora) since 2021. Graduated in Software Engineering and Telecommunications, with more than 25 years of experience in the area and Experienced Integrator certified by **Ericsson**

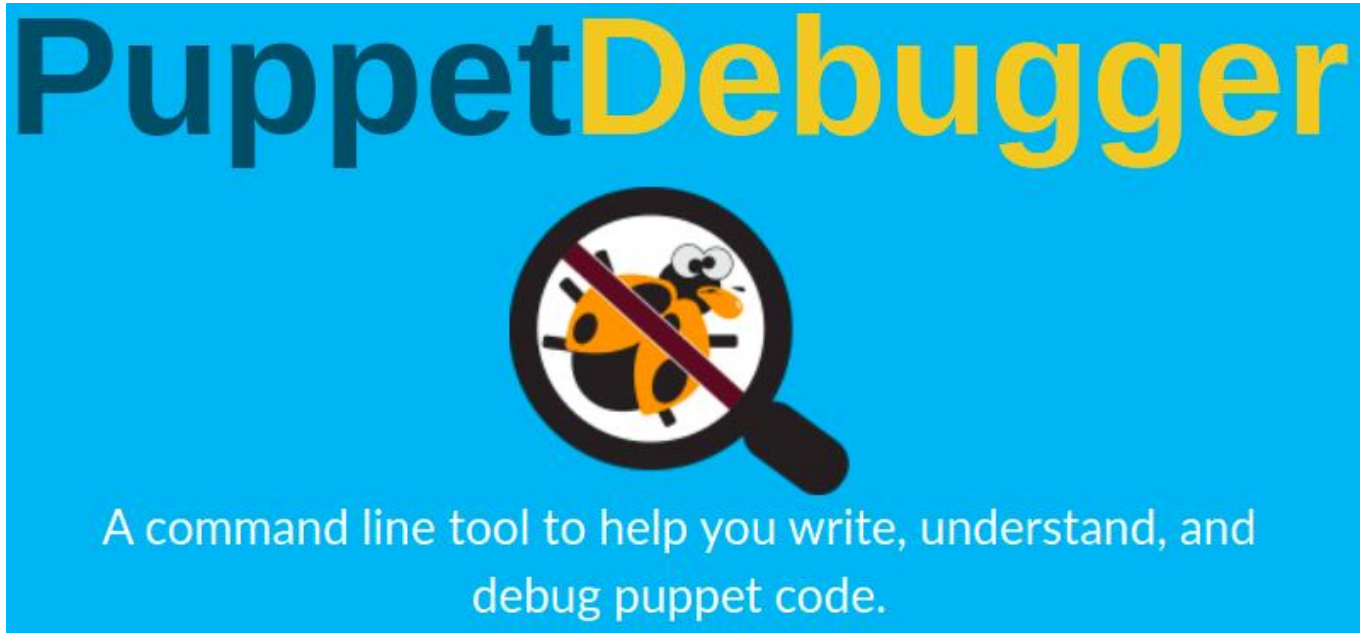
Email : roberto.nogueira@encora.com
LinkedIn: https://www.linkedin.com/public-profile/settings?trk=d_flagship3_profile_self_view_public_profile

DAITAN IS NOW encora

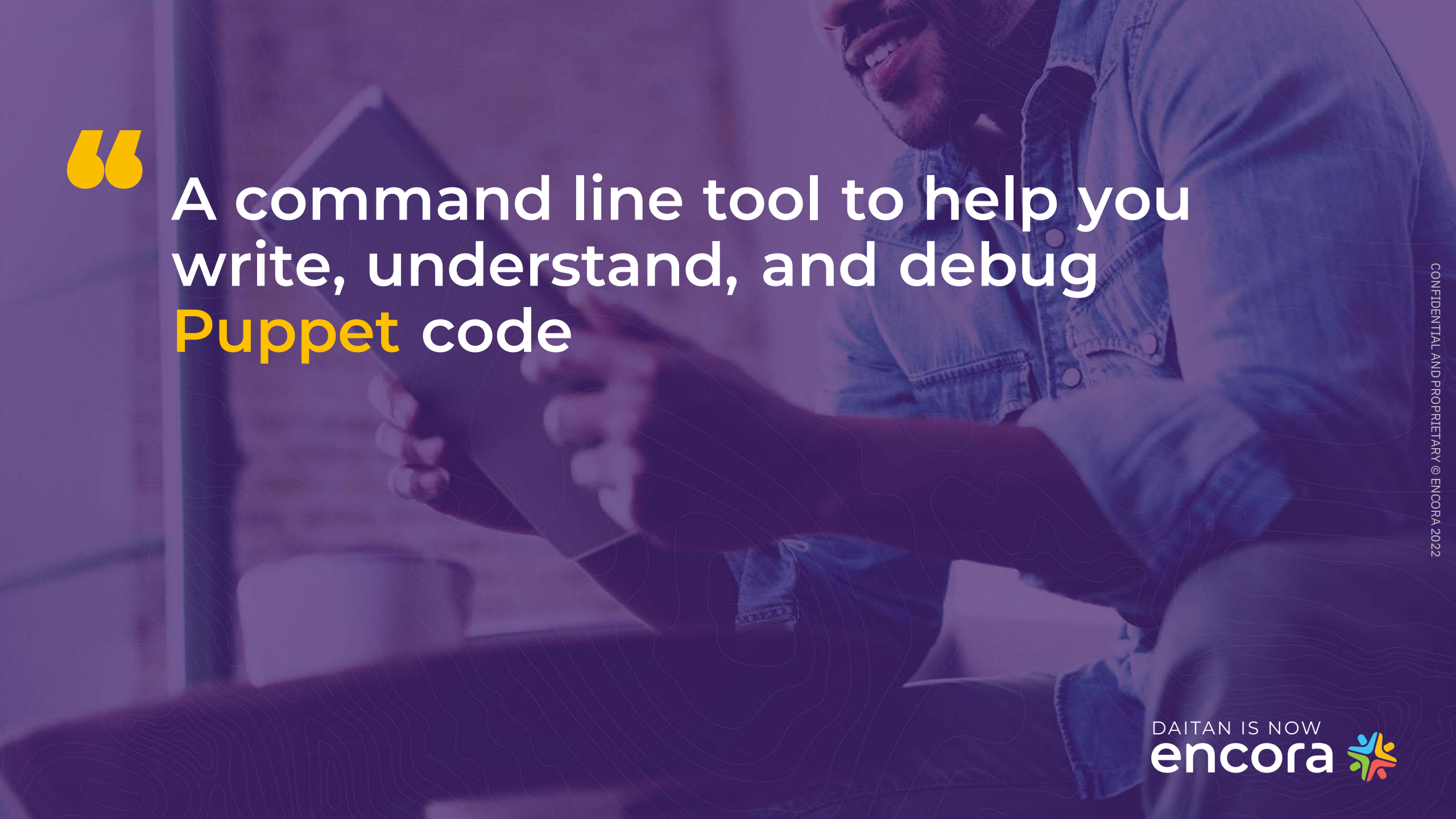
Puppet Debugger
Contents

Puppet Debugger
Objective

It is to introduce a **Pupper Debugger**, a command line tool to help you write, understand, and debug Puppet code.
In order to achieve that the Puppet and its language is revised.



Demo is an Online Puppet Debugger.

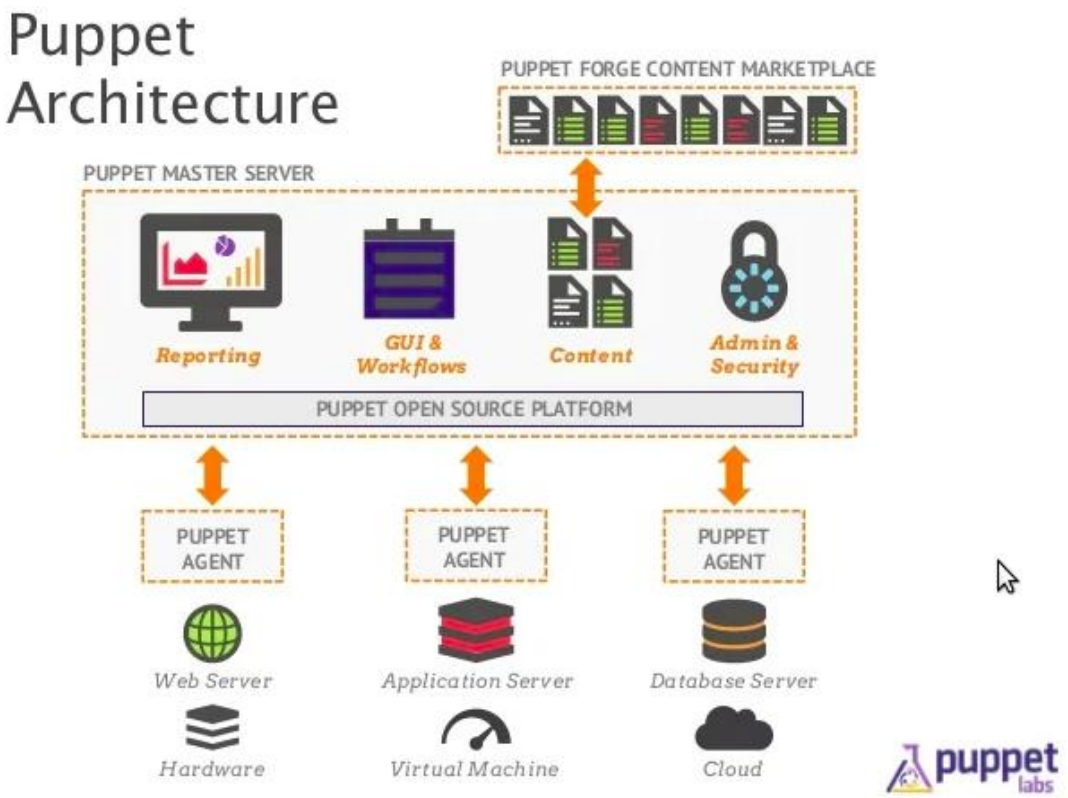


> A command line tool to help you write, understand, and debug Puppet code

DAITAN IS NOW encora

Puppet
Architecture

Puppet is a software configuration management tool which includes its own declarative language to describe system configuration.



DAITAN IS NOW encora

Puppet
Installation

DAITAN IS NOW encora

Install Puppet and Puppet Debugger

```
$ docker run --name=debian11 -t -i --rm debian:11 bash

$ update
$ apt install systemctl wget lsb-release -y
$ wget https://apt.puppetlabs.com/puppet7-release-bionic.deb
$ dpkg -i puppet7-release-bionic.deb
$ apt update
$ apt-get install puppet-agent -y
$ echo 'export PATH=$PATH:/opt/puppetlabs/puppet/bin' >> ~/.bashrc
$ source ~/.bashrc
$ gem install --no-document puppet-debugger
```

Puppet relevant commands

```
# print puppet configuration, usually has to be execute with sudo
$ puppet config print
  :
# set log level to debug, usually has to be execute with sudo
$ puppet config set log_level debug
  :
```

In order to install **Puppet** the way it is described here **Docker** is required see **Ref.1** for further installation.

Puppet Debugger

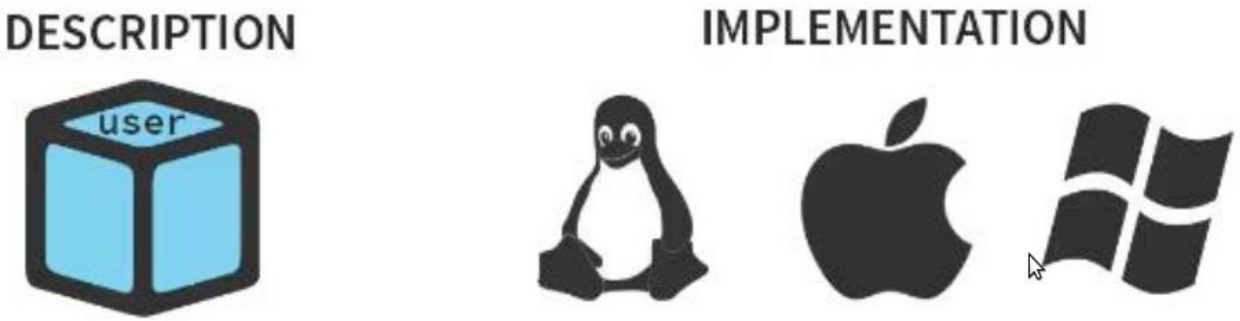
Puppet Language Basics

- Introduction
- Resources
- Classes
- Modules
- Variables
- Metaparameters

Puppet Language Basics

Introduction

Puppet consists of a custom **declarative language** to describe system configuration, which can be either applied directly on the system, or compiled into a catalog and distributed to the target system via **client–server paradigm** (using a REST API), and the agent uses system specific **providers** to enforce the resource specified in the manifests. The resource abstraction layer enables administrators to describe the configuration in high-level terms, such as users, services and packages without the need to specify OS specific commands (such as rpm, yum, apt).



DAITAN IS NOW encora

Puppet Language Basics

Resources

Are the fundamental **building blocks** of the Puppet Language.

Is characteristic of a **node** that needs to be managed overtime.

Resorces Types are e.g. User, Package and Service.

DAITAN IS NOW encora

Puppet Language Basics
Resources

Resource Declaration adds a resource to the catalog and tells Puppet to manage that resource state.

```
resource_type { 'name':
  attribute => value,
}
```

```
file { '/etc/motd':
    ensure  => 'file',
    content => 'Hello World',
    owner   => 'root',
    group   => 'root',
    mode    => '0644',
}
```

DAITAN IS NOW encora

Puppet Language Basics

Resources Examples in Debian

Check the existent resources types

```
$ puppet resource --typesadd
:
file
package
:
```

Check the existent resources **file** type

```
$ puppet resource file '/etc/hosts'
:
file { '/etc/hosts':
  ensure   => 'file',
  content  =>
'{sha256}674a2d0859a6ed81ce1e3513f6ee1f710b47c3b97301b284124d5b95cf566
b47',
  ctime    => '2022-05-21 12:17:40 +0000',
  group    => 0,
  mode     => '0644',
  mtime    => '2022-05-21 12:17:40 +0000',
  owner    => 0,
  provider => 'posix',
  type     => 'file',
}
```

Puppet Language Basics

Resources Examples in Debian

Check the existent resource **service** type

```
$ service --status-all
 [ ? ]  hwclock.sh
 [ - ]  procps

$ puppet resource service 'procps'
service { 'procps':
  ensure   => 'running',
  enable   => 'false',
  provider => 'debian',
}
```

Installing a resources **package** type

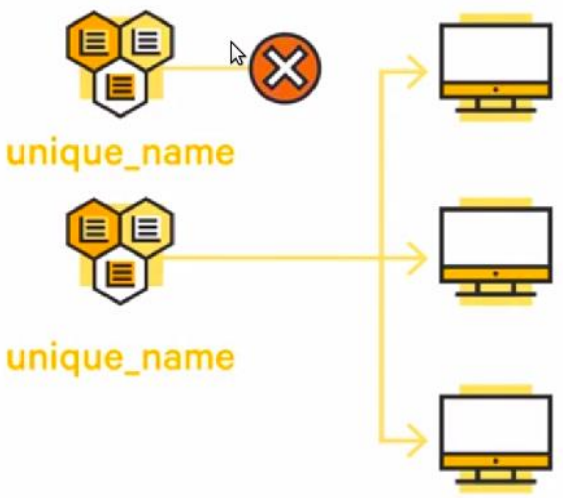
```
$ puppet resource package 'tree' ensure=present
Notice: /Package[tree]/ensure: created
package { 'tree':
  ensure   => 'purged',
  provider => 'apt',
}
```

See also in **Resources** of **Puppet Language Basics** in Puppet channel in YouTube and homepage.

Puppet Language Basics

Classes

Multiple Puppet Resources can be collected and distributed together using **Classes**, by combining Resources into a Class. A Class can configure a large block of functionality.




```
class ssh {
    package { 'openssh-clients':
        ensure => present,
    }
    file { 'etc/ssh/ssh_config':
        ensure  => file,
        content => template('ssh/ssh_config'),
    }
    service { 'sshd':
        ensure => running,
        enable => true,
    }
}
```

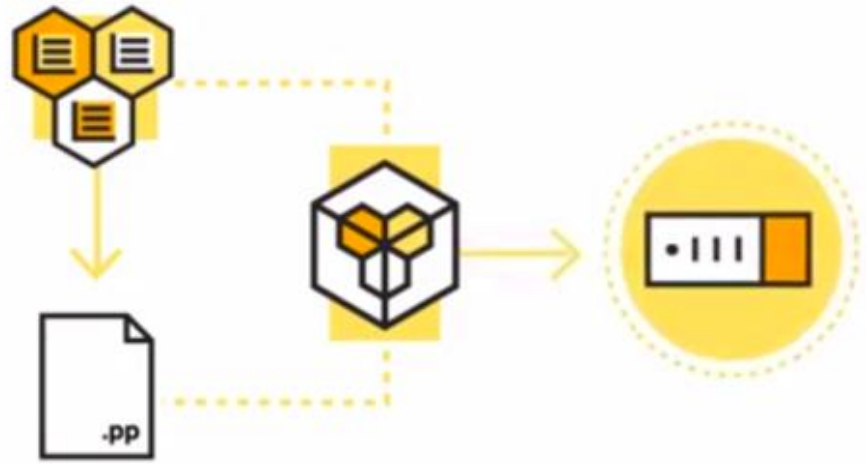
See also in **Classes** of **Puppet Language Basics** in Puppet channel in YouTube and homepage.

DAITAN IS NOW encora

Puppet Language Basics

Modules

Modules are one of the most basic building blocks of the Puppet Language. They are used to manage o specific task or an application such as installing or configuring a piece of software.



Puppet Language Basics

Modules

Module is simply a directory tree that contains the files needed for a given configuration. This directory structure allows nodes to discover and loads Classes, Resources Types and more.

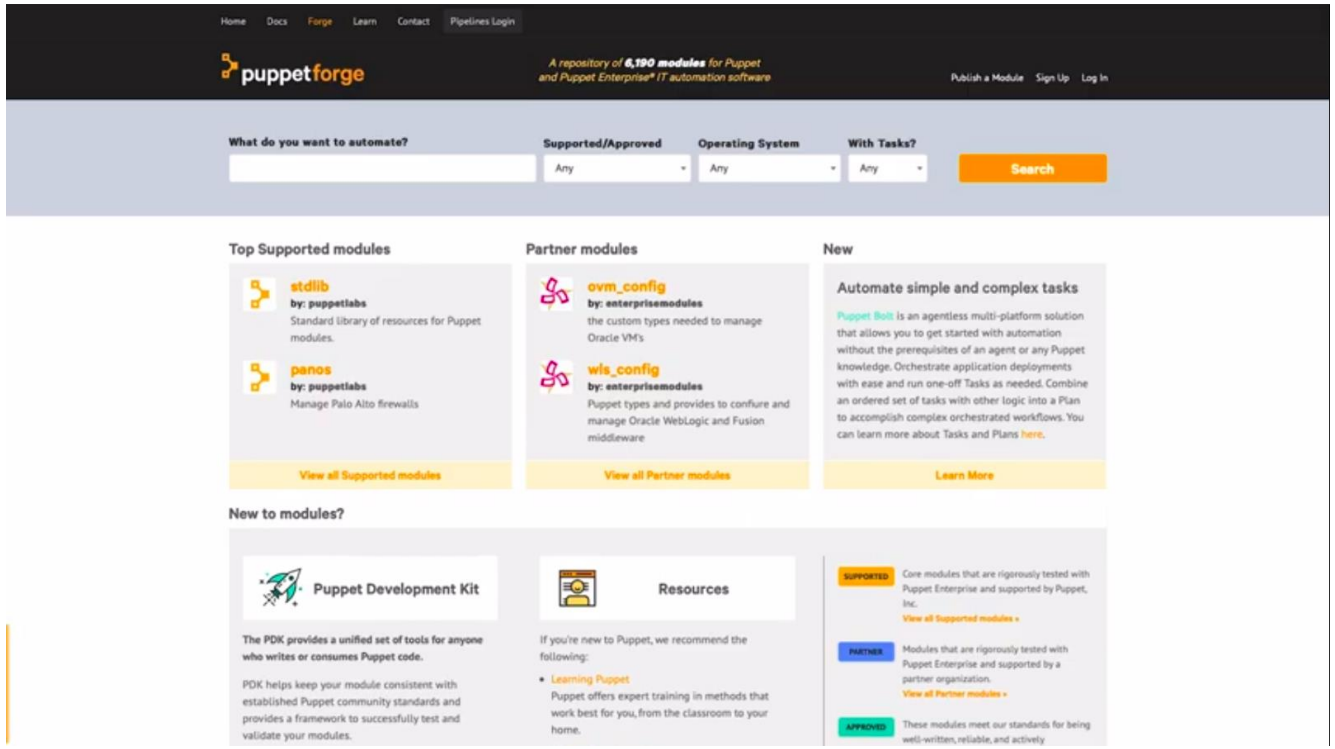
```
module-name/
|- manifests
  |- init.pp
|- files
|- templates
|- examples
|- spec
|- metadata.json
```

module-name is the name of the top's directory tree. If a Class is defined in a Module you can declare that name in any Manifest. When Puppet Master compiles the catalog it wil dynamic load the Manifest that contain the Class definition.

Puppet Language Basics
Modules

DAITAN IS NOW encora

There are thousand of Modules available for downloading at **Puppet Forge**, written by Puppet developers and community members. There are compatible with multi-platforms.



Puppet Language Basics
Modules

Puppet Module relevant commands

```
# list puppet modules and its path, usually has to be execute with sudo
$ puppet module list
:
$ puppet config print modulepath
:
# install puppet modules, usually has to be execute with sudo
$ puppet module install stdlib
:
```

See also in **Modules** of **Puppet Language Basics** in Puppet channel in YouTube and homepage.

Puppet Language Basics

Variables

Variables exists in order to be interpolated in strings to avoid rework in case the value is updated.

```
file { 'path/to/file/share/directory1':
   ensure => directory,
}

file { 'path/to/file/share/directory2':
   ensure => directory,
}

file { 'path/to/file/share/directory3':
   ensure => directory,
}
```

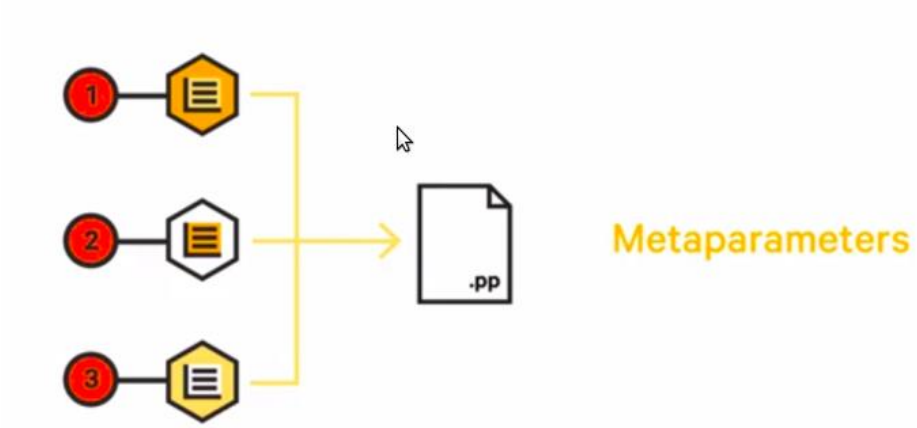
```
$file_share_path = 'new/path/to/file/share'

file { "${file_share_path}/directory1":
   ensure => directory,
}

file { "${file_share_path}/directory2":
   ensure => directory,
}

file { "${file_share_path}/directory3":
   ensure => directory,
}
```

See also in **Variables** of **Puppet Language Basics** in Puppet channel in YouTube and homepage.

Puppet Language Basics

Metaparameters

When a Puppet manifest is loaded the resources inside the manifest are managed the order they are declared. But when you need a group of resources to be managed in different or specific order you must explicit declare the relationships there where the **Metaparameters** come in. **Metaparameters** are special attributes that can be used in any resource type, and specify how your system act to the resource rather than mapping directly to the system state.



DAITAN IS NOW encora

Puppet Language Basics

Metaparameters

The Puppet Language uses four Metaparameters to establish relationships.

- **'before'** causes a resource to be applied before the target resource.
- **'require'** causes a resource to be applied after the target resource.
- **'notify'** causes a resource to be applied *before* the target resource, but will refresh only if the notifying resource changes.
- **'subscribe'** causes a resource to be applied *after* the target resource, again refreshing only if the target resource changes.

```
file { '/etc/ssh/sshd_config':
   ensure  => file,
   mode    => 600,
   source  => 'puppet:///modules/sshd/sshd_config',
   require => Package['openssh-server'],
}

package { 'openssh-server':
   ensure  => present,
   before  => File['/etc/ssh/sshd_config'],
}

service { 'sshd':
   ensure     => running,
   enable     => true,
   subscribe  => File['etc/ssh/sshd_config],
}
```

See also in **Metaparameters** of **Puppet Language Basics** in Puppet channel in YouTube and homepage.

Puppet Debugger

Features

- Puppet Code and Language Evaluator
- Fully Featured **REPL**
- Mock real or fake systems using **facterdb**
- Works with puppet **4.x**, **5.x**, **6.x** and **7.x**
- Easy Gem installation
- Easy Code Playback
- Set Breakpoints
- List all functions, datatypes, types
- Plugin system for additional functionality
- Test snippets of code in realtime

Puppet Debugger

Puppet Debugger Commands

The debugger comes with many commands that allow you to inspect the catalog, your code or reference items currently in scope. All commands are plugins that can process, modify or perform some action on the input and/or output of the debugger.

```
1:>> commands
 Context
   environment          Show the current environment name
   reset                Reset the debugger to a clean state.
   whereami             Show code surrounding the current context.

 Editing
   play                 Playback a file or URL as input.

 Environment
   datatypes            List all the datatypes available in the environment.
   functions            List all the functions available in the environment.
   types                List all the types available in the environment.

 Help
   commands             List all available commands, aka. this screen
   exit                 Quit Puppet Debugger.
   help                 Show the help screen with version information.

 Node
   classification       Show the classification details of the node.
   facterdb_filter      Set the facterdb filter
   facts                List all the facts associated with the node.

 Scope
   classes              List all the classes current in the catalog.
   krt                  List all the known resource types.
   resources            List all the resources current in the catalog.
   set                  Set the a puppet debugger config
   vars                 List all the variables in the current scopes.

 Tools
   benchmark            Benchmark your Puppet code.
```

DAITAN IS NOW encora

Puppet Debugger Commands

Play (Playback Support)

Playing back **files** or **urls** simply loads the content into the debugger session.
https://gist.githubusercontent.com/logicminds/f9b1ac65a3a440d562b0/raw

controls.pp — Raw

```
1  $var1 = 'test'
2  file{"/tmp/${var1}.txt": ensure => present, mode => '0755'}
3  vars
```

```
1:>> play https://gist.githubusercontent.com/logicminds/f9b1ac65a3a440d562b0/raw
>> $var1 = 'test'
 => "test"
>> file{"/tmp/${var1}.txt": ensure => present, mode => '0755'}
 => Puppet::Type::File {
                "checksum" => "sha256",
                  "ensure" => "present",
                   "links" => "manage",
               "loglevel" => "notice",
              "max_files" => 0,
                    "mode" => "0755",
                    "name" => "/tmp/test.txt",
                    "path" => "/tmp/test.txt",
               "provider" => "posix",
                   "purge" => false,
```

DAITAN IS NOW encora

Puppet Debugger Commands

Play (Web Based Playback Support)

If using the web based debugger you can playback a shared url which would start a debugger session and then load the content from the url or parameter.

Examples:

Urls
https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/64f0fe9f64339f18f097a9f42acd6276
https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4

Simple commands
https://demo.puppet-debugger.com/play?content=vars

Content Parameter
https://demo.puppet-debugger.com/play?content=md5('dafsdfdsfasdfds')

The demo site is a web application that runs a local instance of the debugger per session.

So create a link, test it and share it.

Puppet Debugger Commands

Play (Web Based Playback Support)

DAITAN IS NOW encora

https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4

repl_test.pp — Raw

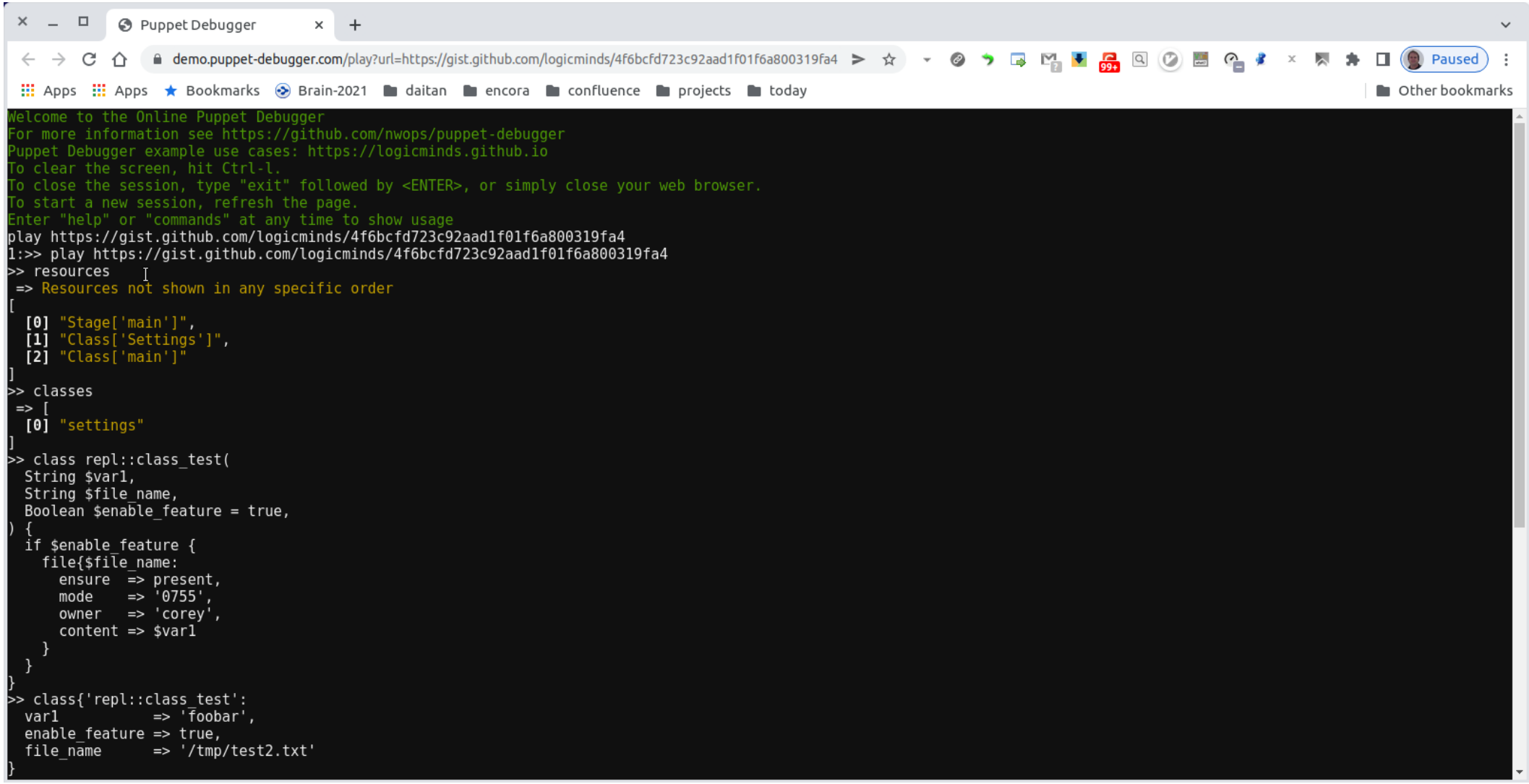
```
resources
classes
class repl::class_test(
  String $var1,
  String $file_name,
  Boolean $enable_feature = true,
) {
  if $enable_feature {
    file{$file_name:
      ensure  => present,
      mode    => '0755',
      owner   => 'corey',
      content => $var1
    }
  }
}
class{'repl::class_test':
  var1           => 'foobar',
  enable_feature => true,
  file_name      => '/tmp/test2.txt'
}
resources
classes
```

https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4

Puppet Debugger Commands

Play (Web Based Playback Support)

DAITAN IS NOW encora



Puppet Debugger Commands
FacterDB

DAITAN IS NOW encora

The **puppet-debugger** internally leverages the facterdb gem to load pre-cached facts into the debugger session. You can tell the debugger to use a different set of facts from a different kernel, **OS version**, facter version or any custom combination that facterdb supports. This is all controlled via the facterdb-filter.

```
puppet debugger --facterdb-filter 'facterversion=/^3.6./ and operatingsystem=windows' --test -e '$os[family]'

puppet debugger --facterdb-filter 'facterversion=/^3.6./ and osfamily=RedHat' --test -e '$os[family]'

puppet debugger --facterdb-filter 'facterversion=/^3.6./ and osfamily=Darwin' --test -e '$os[family]'
```

```
{
  "name": "macbook-pro-10.domain",
  "values": {
    "custom_datacenter_fact": "iceland",
    "puppetversion": "5.3.2",
    "architecture": "x86_64",
    "kernel": "Darwin",
    "domain": "domain"
  }
}
```

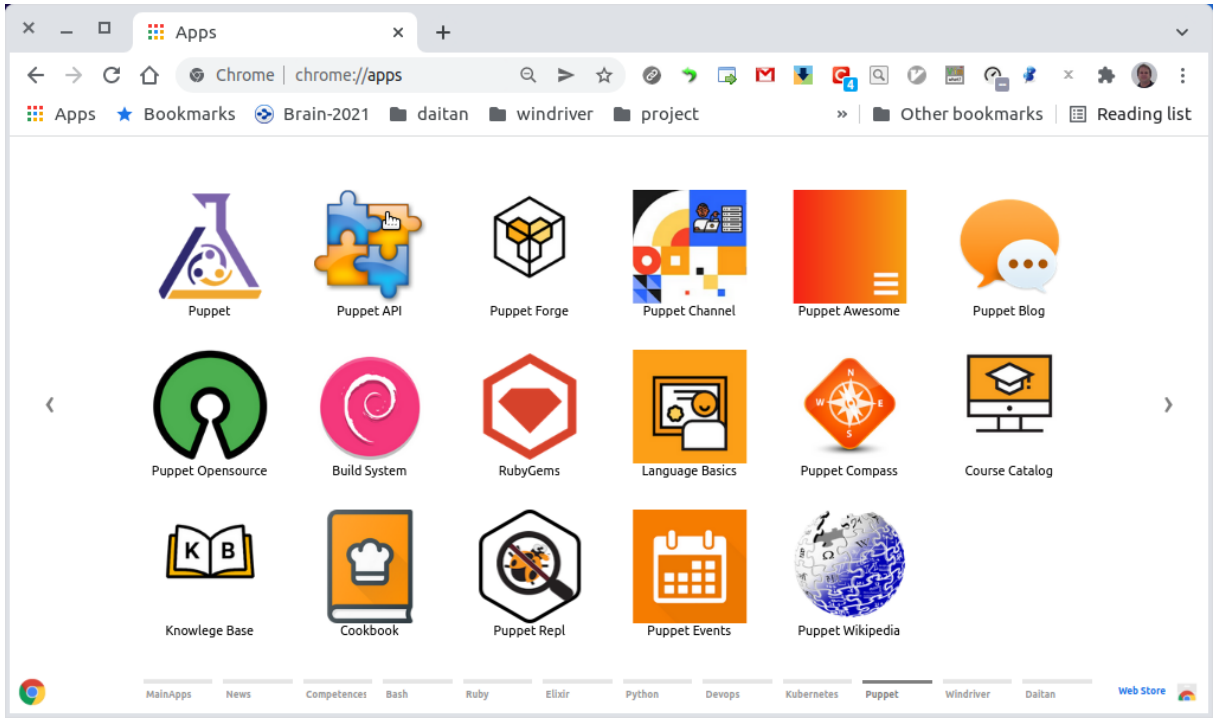
You can force the debugger to use real facts from the **current node** too:

```
puppet debugger --no-facterdb --test -e '$os[family]'
```

Puppet Debugger

Puppet Extensions

Chrome ans VSCode can help you stay productive and get more out of your browser and code editor.



Puppet - Chrome apps for Puppet configuration management.
For futher info about Chrome Apps check Create and publish custom Chrome apps & extensions.

puppet.puppet-vscode - Puppet Visual Studio Code Extension.
For further extensions check VSCode marketplace.

DAITAN IS NOW encora

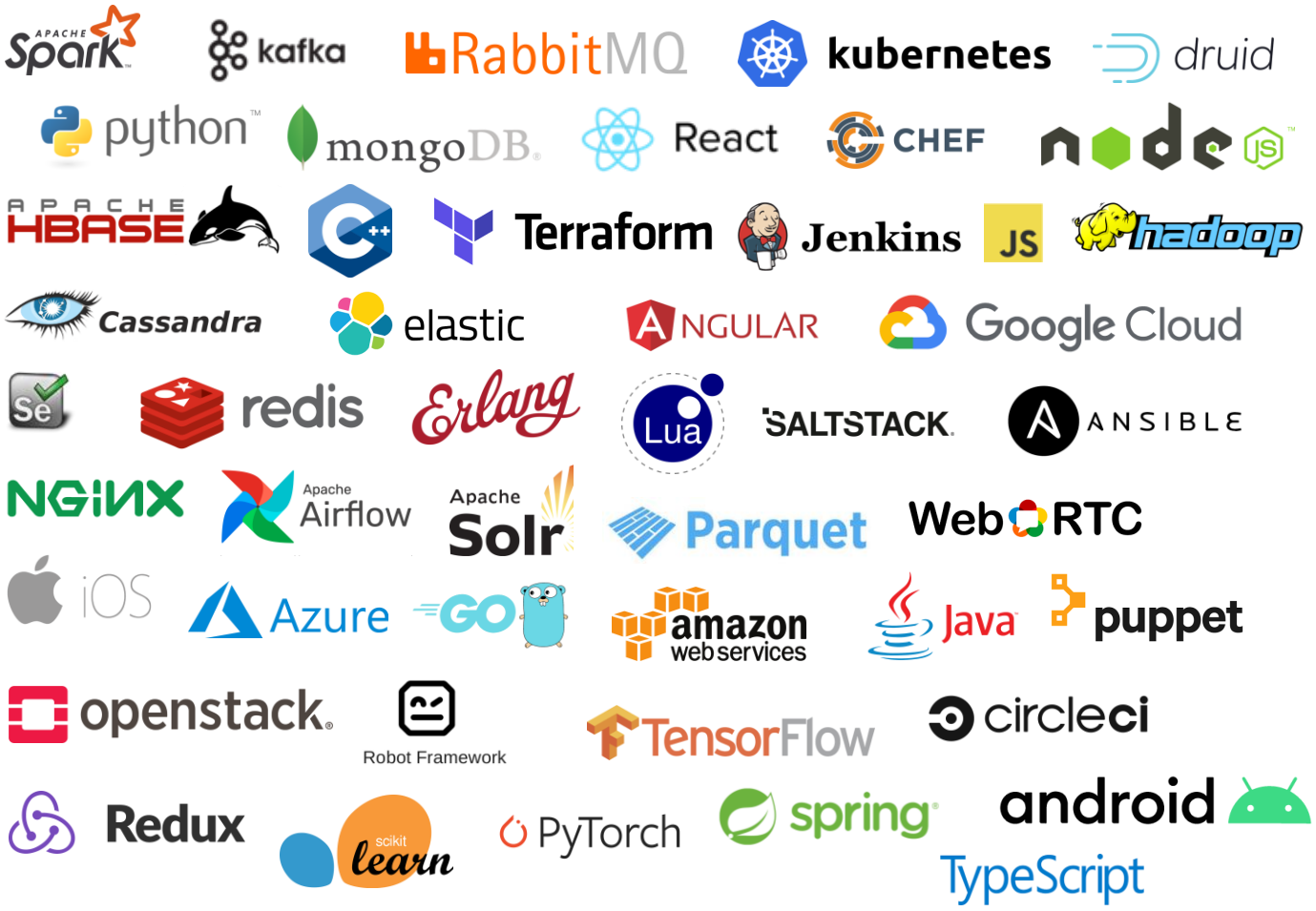
Puppet Debugger
References

1. Puppet in Wikipedia
2. Docker Installation
3. Puppet Language Basics
4. Puppet Forge
5. Puppet Debugger
6. Using the Puppet Debugger for Lightweight Exploration -Corey Osman
7. Puppet VS Code Tips and tricks
8. Puppet YouTube Channel
9. Puppet Cookbook
0. Awesome Puppet

Accelerate with Confidence

Experience with Hundreds of tools and Platforms

DAITAN IS NOW encora



Our experience with software development tools and technologies accelerates deliverables and remove risks from schedules and budgets